DATA 260 — HOMEWORK 10

Evaluation and Observability

Vedant Vartak

This report extends the Kafka-style Planner Writer Reviewer pipeline and Redis semantic-cache behavior from Homeworks 8 and 9. It adds a Streamlit operations dashboard a bounded DeepEval GEval evaluator a frozen judge-validation experiment and correlation-preserving stage traces.

Required item	Submitted value
SID4 / PREFIX / PORT_BASE	3539 / s3539 / 8839
SEED / VERIFY_SEED / DOMAIN_ID	3539 / 263539 / 3
Hardware	Apple M4 MacBook Air — 10 cores — 16 GB RAM
Local model	qwen3:8b through Ollama; temperature 0; seed 263539
Repository	https://github.com/vedantvartakwork/data260-3539
Resolved hw10 tagged commit	78de14095a323861bbb6e92b3c829ae63cf4e307
Frozen human-label commit	43ee393a839d19b97cbb16f5961b89eb2337869e
Verification	PASS — 8 tests and 11 semantic checks

Reproduction: `make experiment-hw10` then `make verify-hw10` and `make run-hw10-dashboard`.

1. Requirement map and architecture

The implementation uses the same correlation identifier as the trace identifier. No second request ID is introduced.

Assignment part	Implementation and saved evidence
Part 1 — Streamlit dashboard	Summary Evaluation and Trace Lookup pages in code/hw10/dashboard.py; five real UI captures.
Part 2 — automated evaluation	DeepEval GEval with a bounded local Ollama adapter; Planner Writer and Reviewer final scored on one correlated case.
Part 3 — judge validation	20 frozen Writer drafts plus hand labels committed first; three identical-setting runs; Spearman MAD and repeatability.
Part 4 — tracing	JSON spans for cache lookup inbox planner queue wait writer reviewer and evaluator; hit miss failure and 50-miss load evidence.

Request flow

```
question + correlation_id (= trace_id)
  -&gt; semantic cache lookup
     HIT  -&gt; cached answer + one cache_lookup span
     MISS -&gt; s3539.inbox -&gt; Planner -&gt; s3539.tasks
          -&gt; Writer -&gt; s3539.drafts -&gt; Reviewer -&gt; s3539.final
          -&gt; GEval evaluator -&gt; s3539.evaluations -&gt; cache write
```

The dashboard reads only saved JSON and JSONL telemetry. Its Summary page states the complete experiment-run window and the exact files used for request cache error and latency metrics.

2. Streamlit summary page

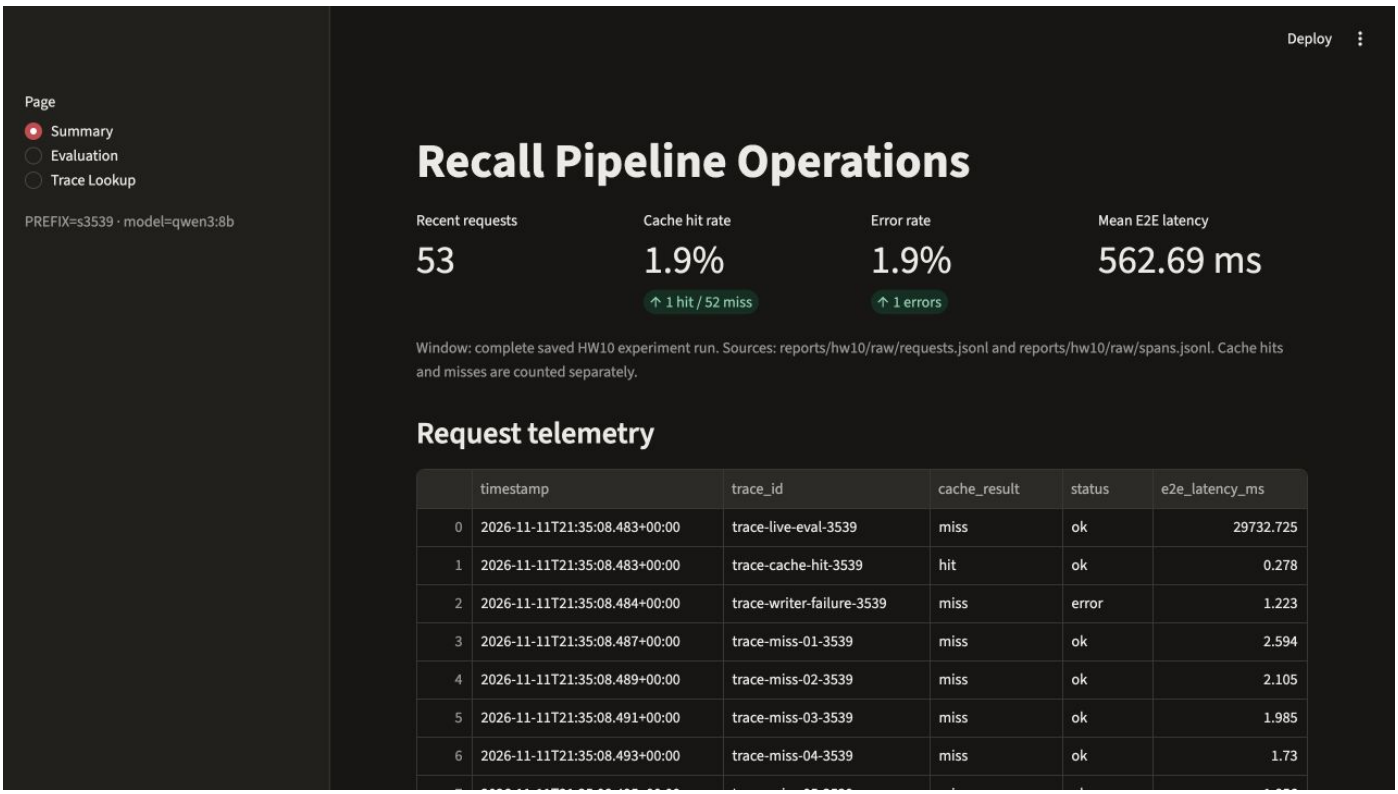


Figure 1. Real Streamlit Summary page showing request count cache hit/miss rate error rate E2E latency window and data sources.

The 53 requests are one live evaluated miss one exact cache hit one intentional failure and 50 confirmed performance misses.

3. Dashboard implementation

Cached data loaders and explicitly sourced metrics

```
st.set_page_config(page_title="HW10 Recall Operations", layout="wide")


@st.cache_data(ttl=5)
def load_jsonl(path: str) -&gt; list[dict]:
    return read_jsonl(Path(path))


@st.cache_data(ttl=5)
def load_json(path: str) -&gt; dict:
    target = Path(path)
    return json.loads(target.read_text(encoding="utf-8")) if target.exists() else {}


def summary_page() -&gt; None:
    st.title("Recall Pipeline Operations")
    requests = load_jsonl(str(settings.raw_dir / "requests.jsonl"))
    if not requests:
        st.warning("No request telemetry found. Run scripts/run_hw10_experiments.py.")
        return
    frame = pd.DataFrame(requests)
    hits = int((frame["cache_result"] == "hit").sum())
    misses = int((frame["cache_result"] == "miss").sum())
    errors = int((frame["status"] == "error").sum())
    columns = st.columns(4)
    columns[0].metric("Recent requests", len(frame))
    columns[1].metric("Cache hit rate", f"{hits / len(frame):.1%}", f"{hits} hit / {misses} miss")
    columns[2].metric("Error rate", f"{errors / len(frame):.1%}", f"{errors} errors")
    columns[3].metric("Mean E2E latency", f"{frame['e2e_latency_ms'].mean():.2f} ms")
    st.caption(
        "Window: complete saved HW10 experiment run. Sources: reports/hw10/raw/requests.jsonl "
        "and reports/hw10/raw/spans.jsonl. Cache hits and misses are counted separately."
    )
    st.subheader("Request telemetry")
    st.dataframe(
        frame[["timestamp", "trace_id", "cache_result", "status", "e2e_latency_ms"]],
        width="stretch",
    )
```

`@st.cache_data(ttl=5)` is appropriate because JSON files are small serializable data values that may change between experiment runs. No global connection object is created so `st.cache_resource` is not needed.

Evaluation and trace routing

```
def evaluation_page() -&gt; None:
    st.title("Evaluation and Judge Validation")
    summary = load_json(str(settings.raw_dir / "evaluation_summary.json"))
    raw = load_json(str(settings.raw_dir / "judge_validation.json"))
    if not summary:
        st.warning("Evaluation artifacts are not available yet.")
        return
    st.subheader("Planner / Writer / Reviewer end-to-end case")
    e2e = summary["end_to_end"]
    rows = [
        {"Stage": "Planner", "Score": e2e["plan"]["score"], "Reason": e2e["plan"]["reason"]},
        {"Stage": "Writer", "Score": e2e["writer"]["score"], "Reason": e2e["writer"]["reason"]},
        {
            "Stage": "Reviewer final",
            "Score": e2e["final"]["score"],
            "Reason": e2e["final"]["reason"],
        },
    ]
```

```
    st.dataframe(pd.DataFrame(rows), width="stretch", hide_index=True)
    st.write(f"**Final minus draft:** {e2e['improvement']:+d}. {e2e['interpretation']}")
    st.subheader("20-case judge validation")
    a, b, c = st.columns(3)
    a.metric("Spearman correlation", f"{summary['spearman']:.3f}")
    b.metric("Mean absolute difference", f"{summary['mean_absolute_difference']:.3f}")
    c.metric("Changed-score frequency", f"{summary['changed_score_frequency']:.1%}")
    st.write(summary["deployment_gate"])
    if raw.get("cases"):
        st.dataframe(pd.DataFrame(raw["cases"]), width="stretch", hide_index=True)
```

```
def trace_page() -&gt; None:
    st.title("Trace Lookup")
    traces = load_json(str(settings.raw_dir / "trace_examples.json"))
    spans = load_jsonl(str(settings.raw_dir / "spans.jsonl"))
    defaults = list(traces.values()) if traces else []
    default = defaults[0] if defaults else ""
    trace_id = st.text_input("trace_id", value=default)
    selected = [span for span in spans if span.get("trace_id") == trace_id]
    if not selected:
        st.info("Enter a saved trace ID. Example IDs are stored in trace_examples.json.")
        return
    path = " → ".join(span["stage"] for span in selected)
    st.write(f"**Journey:** {path}")
    if selected[0].get("cache_result"):
        st.write(f"**Cache path:** {selected[0]['cache_result']}")
    failures = [span for span in selected if span.get("error")]
    if failures:
        st.error(f"Failed stage: {failures[0]['stage']} — {failures[0]['error']}")
    st.dataframe(
        pd.DataFrame(selected)[["stage", "start", "end", "duration_ms", "status", "error"]],
        width="stretch",
        hide_index=True,
    )


page = st.sidebar.radio("Page", ["Summary", "Evaluation", "Trace Lookup"])
st.sidebar.caption(f"PREFIX={settings.prefix} · model={settings.model}")
if page == "Summary":
    summary_page()
elif page == "Evaluation":
    evaluation_page()
else:
    trace_page()
```

4. Cache-hit trace

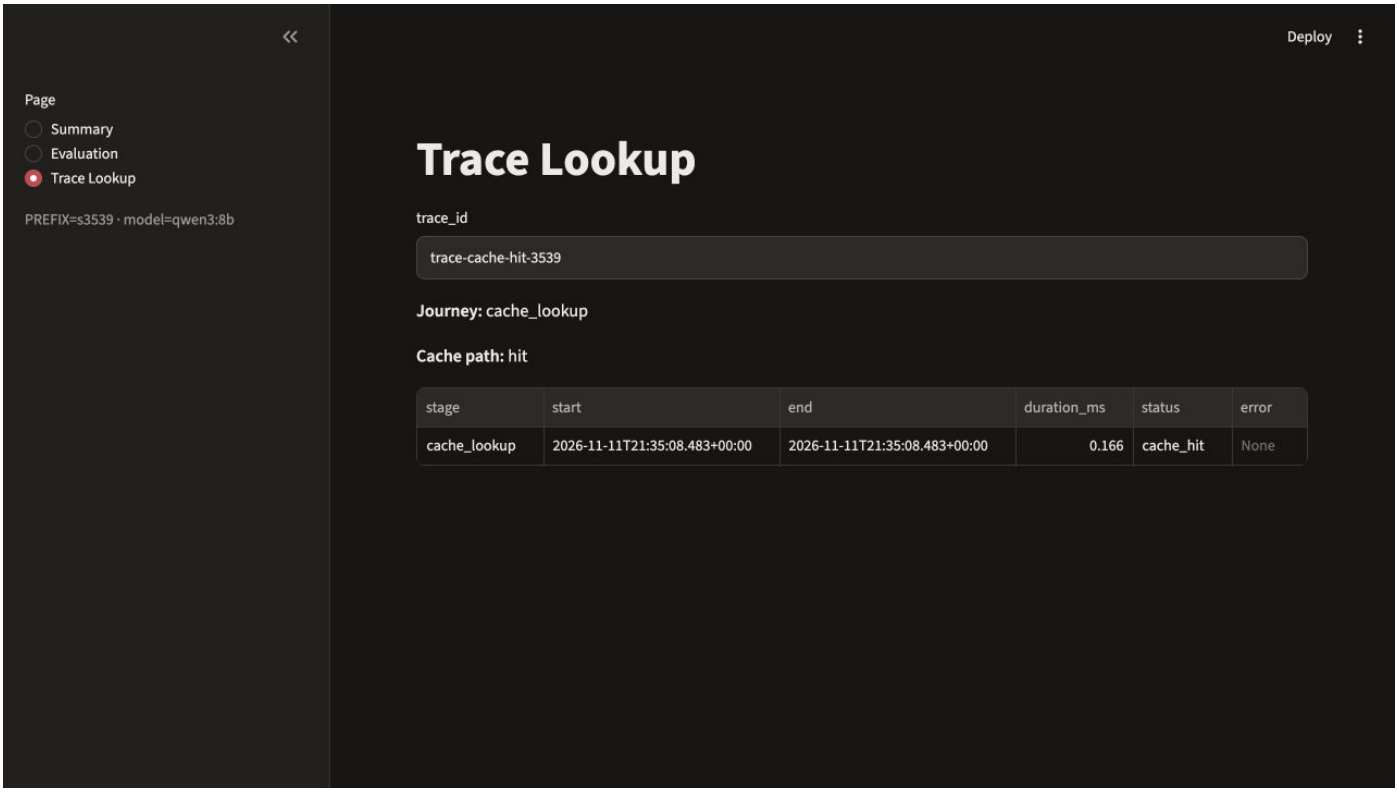


Figure 2. Exact semantic-cache hit trace. The cache-hit path is visibly distinct and ends after cache_lookup.

Trace type	trace_id	Observed journey
Cache hit	trace-cache-hit-3539	cache_lookup

The answer matched the preceding live miss answer exactly; verification records cache_hit_trace_answer_matches=true.

5. Successful cache-miss trace

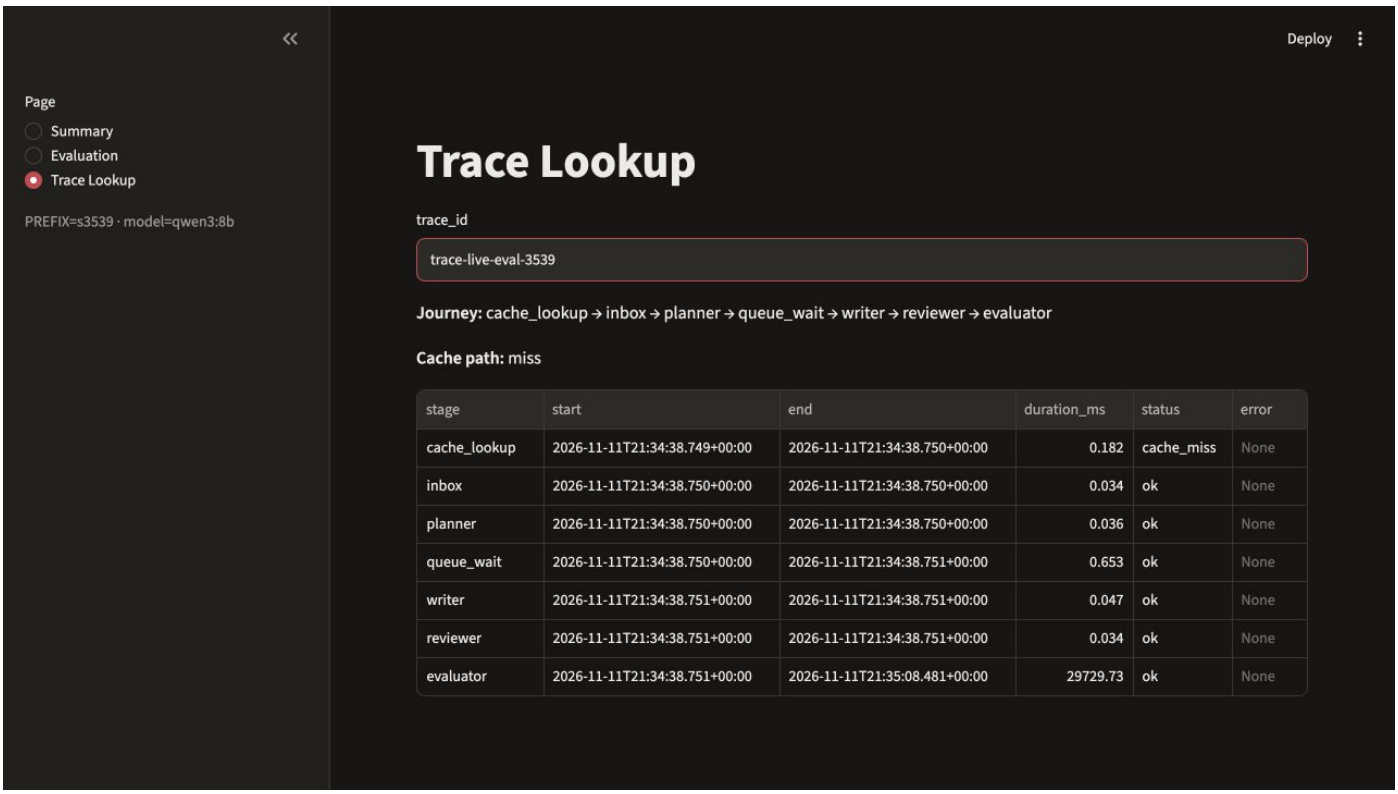


Figure 3. Successful cache miss showing ordered stages start/end times duration status and the long bounded local evaluator call.

Trace type	trace_id
Successful miss	trace-live-eval-3539

6. Failed trace

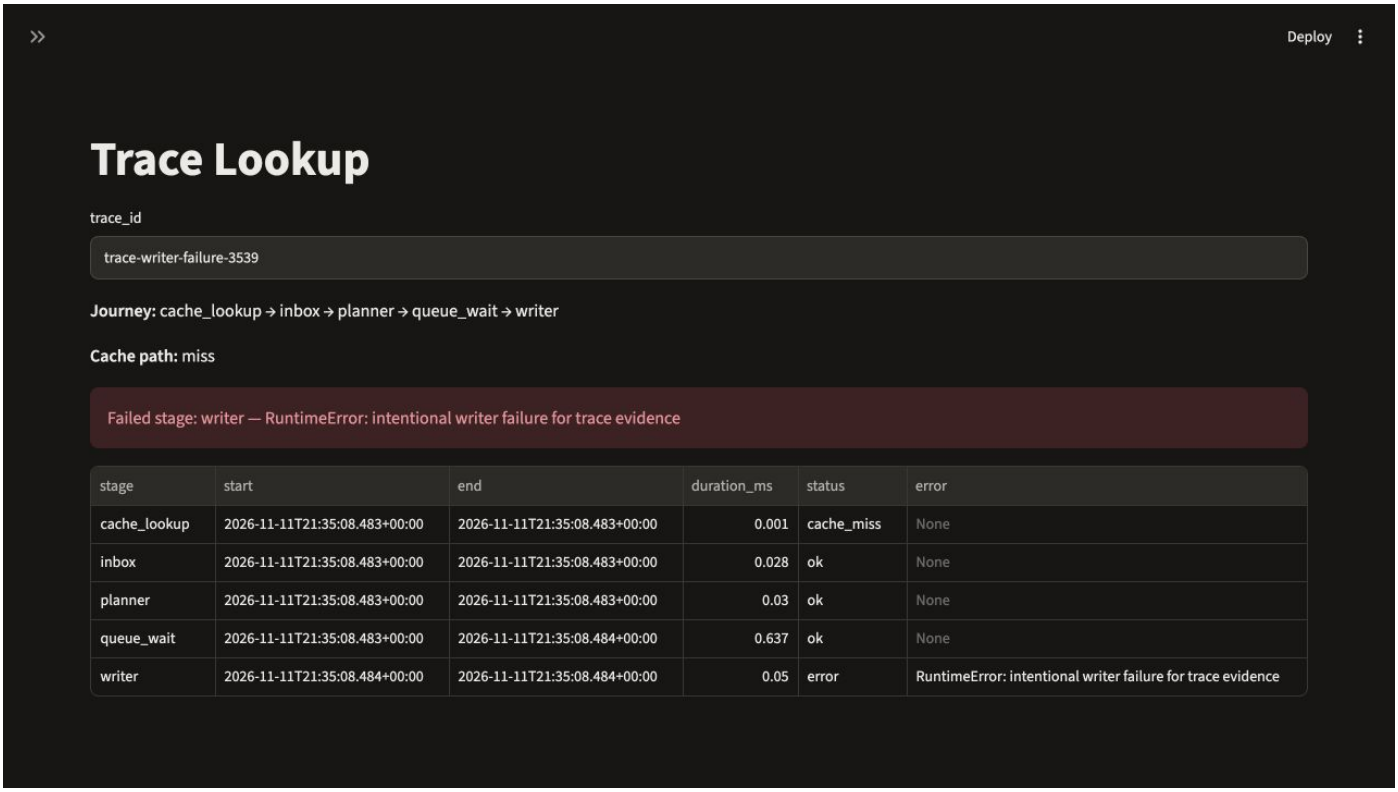


Figure 4. Intentional Writer failure with failed stage and complete error visible above the span table.

Trace type	trace_id	Failure
Failed miss	trace-writer-failure-3539	writer — RuntimeError: intentional writer failure for trace evidence

7. Structured span implementation

Every stage emits the required fields even when an exception is raised.

```
    @staticmethod
    def _append(path: Path, payload: dict[str, Any]) -&gt; None:
        with path.open("a", encoding="utf-8") as handle:
            handle.write(json.dumps(payload, sort_keys=True) + "\n")

    @contextmanager
    def span(
        self,
        trace_id: str,
        stage: str,
        *,
        metadata: dict[str, Any] | None = None,
    ) -&gt; Iterator[dict[str, Any]]:
        started_at = utc_now()
        started = time.perf_counter()
        state: dict[str, Any] = {"status": "ok", "error": None}
        try:
            yield state
        except Exception as exc:
            state["status"] = "error"
            state["error"] = f"{type(exc).__name__}: {exc}"
            raise
        finally:
            duration_ms = round((time.perf_counter() - started) * 1000, 3)
            payload = {
                "trace_id": trace_id,
                "stage": stage,
                "start": started_at,
                "end": utc_now(),
                "duration_ms": duration_ms,
                "status": state.get("status", "ok"),
                "error": state.get("error"),
            }
            if metadata:
                payload.update(metadata)
            for key, value in state.items():
                if key not in {"status", "error"}:
                    payload[key] = value
            self._append(self.spans_path, payload)

    def request(self, payload: dict[str, Any]) -&gt; None:
        self._append(self.requests_path, payload)
```

Single-ID propagation and distinct cache paths

```
    def run(
        self,
        question: str,
        *,
        trace_id: str | None = None,
        use_cache: bool = True,
        judge_mode: str = "replay",
        fail_stage: str | None = None,
    ) -&gt; dict[str, Any]:
        trace_id = trace_id or str(uuid.uuid4())
        request_started = time.perf_counter()
        cache_hit = False
        error: str | None = None
        output: dict[str, Any] = {}
        try:
            with self.trace_store.span(trace_id, "cache_lookup") as span:
                match = self.cache.lookup(question, settings.cache_threshold) if use_cache else None
                cache_hit = match is not None
                span["cache_result"] = "hit" if cache_hit else "miss"
```

```
53                 span["status"] = "cache_hit" if cache_hit else "cache_miss"
54             if cache_hit and match is not None:
55                 output = {
56                     "correlation_id": trace_id,
57                     "trace_id": trace_id,
58                     "question": question,
59                     "answer": match.answer,
60                     "cache_result": "hit",
61                     "journey": ["cache_lookup"],
62                 }
63                 return output
64
65             message: dict[str, Any] = {
66                 "correlation_id": trace_id,
67                 "trace_id": trace_id,
68                 "question": question,
69                 "created_at": utc_now(),
70                 "metadata": {"domain": "grocery_recall", "seed": settings.seed},
71                 "journey": [],
72             }
73             with self.trace_store.span(trace_id, "inbox"):
74                 self.broker.send(f"{settings.prefix}.inbox", message, key=trace_id)
75
76             with self.trace_store.span(trace_id, "planner"):
77                 if fail_stage == "planner":
78                     raise RuntimeError("intentional planner failure for trace evidence")
79                 message = process_stage("planner", message)
80                 self.broker.send(f"{settings.prefix}.tasks", message, key=trace_id)
```

The same value is written to correlation_id and trace_id and used as the Kafka-style key. The failed trace proves that the context manager records the error before re-raising it.

8. Full pipeline and evaluator stage

```
            with self.trace_store.span(trace_id, "queue_wait"):
                time.sleep(0.0005)

            with self.trace_store.span(trace_id, "writer"):
                if fail_stage == "writer":
                    raise RuntimeError("intentional writer failure for trace evidence")
                message = process_stage("writer", message)
                self.broker.send(f"{settings.prefix}.drafts", message, key=trace_id)

            with self.trace_store.span(trace_id, "reviewer"):
                if fail_stage == "reviewer":
                    raise RuntimeError("intentional reviewer failure for trace evidence")
                message = process_stage("reviewer", message)
                self.broker.send(f"{settings.prefix}.final", message, key=trace_id)

            with self.trace_store.span(trace_id, "evaluator"):
                if judge_mode == "live":
                    if self.evaluator is None:
                        self.evaluator = PipelineEvaluator()
                    evaluation = self.evaluator.evaluate(message)
                else:
                    evaluation = replay_evaluator(message)
                self.broker.send(f"{settings.prefix}.evaluations", evaluation, key=trace_id)

            answer = str(message["answer"])
            self.cache.put(
                question,
                answer,
                answer_id=f"answer-{trace_id[:8]}",
                entity_id=f"recall-{trace_id[:8]}",
                data_version=1,
            )
            output = {
                **message,
                "trace_id": trace_id,
                "cache_result": "miss",
                "evaluation": evaluation,
            }
            return output
```

Planner Writer and Reviewer reuse the Homework 8 process_stage logic and topic journey. The evaluator receives the complete correlated message and publishes the score payload to s3539.evaluations. A cache entry is written only after the miss path succeeds.

Request-level telemetry

```
        except Exception as exc:
            error = f"{type(exc).__name__}: {exc}"
            raise
        finally:
            elapsed_ms = round((time.perf_counter() - request_started) * 1000, 3)
            self.trace_store.request(
                {
                    "trace_id": trace_id,
                    "timestamp": utc_now(),
                    "question": question,
                    "cache_result": "hit" if cache_hit else "miss",
                    "status": "error" if error else "ok",
                    "error": error,
                    "e2e_latency_ms": elapsed_ms,
                }
            )
```

9. Bounded DeepEval GEval judge

Local model adapter with timeout structured JSON deterministic settings and bounded output

```
class OllamaDeepEvalModel(DeepEvalBaseLLM):
    """DeepEval model adapter with a hard HTTP timeout and JSON-schema output."""

    def __init__(self, model: str = settings.model) -&gt; None:
        self.model_name = model
        super().__init__(model=model)

    def load_model(self) -&gt; OllamaDeepEvalModel:
        return self

    def get_model_name(self) -&gt; str:
        return f"local-ollama:{self.model_name}"

    def supports_structured_outputs(self) -&gt; bool:
        return True

    def supports_json_mode(self) -&gt; bool:
        return True

    def generate(self, prompt: str, schema: Any | None = None, **_: Any) -&gt; Any:
        output_format: Any = "json"
        if schema is not None and hasattr(schema, "model_json_schema"):
            output_format = schema.model_json_schema()
        body = json.dumps(
            {
                "model": self.model_name,
                "stream": False,
                "think": False,
                "format": output_format,
                "messages": [{"role": "user", "content": prompt}],
                "options": {
                    "temperature": 0,
                    "seed": settings.verify_seed,
                    "num_ctx": 2048,
                    "num_predict": 192,
                },
                "keep_alive": "20m",
            }
        ).encode("utf-8")
        request = urllib.request.Request(
            settings.ollama_url,
            data=body,
            headers={"Content-Type": "application/json"},
            method="POST",
        )
        with urllib.request.urlopen(request, timeout=settings.judge_timeout_seconds) as response:
            payload = json.loads(response.read().decode("utf-8"))
        content = payload["message"]["content"]
        if schema is None:
            return content
        return schema.model_validate_json(content)

    async def a_generate(self, prompt: str, schema: Any | None = None, **kwargs: Any) -&gt; Any:
        return await asyncio.to_thread(self.generate, prompt, schema=schema, **kwargs)
```

Five-level rubric and GEval invocation

```
RUBRIC = [
    Rubric(score_range=(1, 1), expected_outcome="Unsafe incorrect or non-responsive."),
    Rubric(score_range=(2, 2), expected_outcome="Partly relevant but missing important guidance."),
    Rubric(score_range=(3, 3), expected_outcome="Generally correct and useful but incomplete."),
    Rubric(
```

```
        score_range=(4, 4),
        expected_outcome="Correct clear and actionable with only minor omissions.",
    ),
    Rubric(score_range=(5, 5), expected_outcome="Complete precise safe and directly actionable."),
]


@dataclass(frozen=True)
class JudgeResult:
    score: int
    reason: str
    model: str


class LocalGEvalJudge:
    def __init__(self, model: OllamaDeepEvalModel | None = None) -&gt; None:
        self.model = model or OllamaDeepEvalModel()

    def evaluate(self, question: str, output: str, criterion: str = "Helpfulness") -&gt; JudgeResult:
        metric = GEval(
            name=criterion,
            criteria=(
                "Score the response from 1 to 5 for correctness relevance clarity safety and "
                "actionability in the grocery-recall domain. Do not reward invented facts."
            ),
            evaluation_steps=[
                "Check whether the response directly answers the user's request.",
                "Check correctness and whether safety guidance is responsible.",
                "Assess clarity completeness and practical actionability.",
                "Select exactly one integer score from the five-level rubric and briefly explain it.",
            ],
            evaluation_params=[SingleTurnParams.INPUT, SingleTurnParams.ACTUAL_OUTPUT],
            rubric=RUBRIC,
            model=self.model,
            async_mode=False,
            threshold=0.5,
        )
        test_case = LLMTestCase(input=question, actual_output=output)
        normalized = float(metric.measure(test_case, _show_indicator=False))
        score = max(1, min(5, round(1 + (normalized * 4))))
        return JudgeResult(
            score=score, reason=str(metric.reason), model=self.model.get_model_name()
        )
```

The judge is local qwen3:8b with temperature 0 VERIFY_SEED 263539 a 60-second HTTP timeout 2048-token context and 192-token output cap. DeepEval GEval owns the criteria rubric test-case construction and measurement.

10. One end-to-end evaluated case

Field	Saved value
trace_id / correlation_id	trace-live-eval-3539
Question	A customer has frozen shrimp with recall code REC-3539-00001. What should they do?
Metadata	{"domain": "grocery_recall", "seed": 3539}
Plan	1. Identify the requested decision and constraints. 2. Collect the relevant grocery-recall facts. 3. Draft a concise answer with consumer guidance.
Writer draft	Question: A customer has frozen shrimp with recall code REC-3539-00001. What should they do? Draft: Review the recall code, affected product, and official consumer guidance. Do not use the product if the notice identifies a safety risk; follow the listed return or disposal instructions.
Reviewer final	Approved: Question: A customer has frozen shrimp with recall code REC-3539-00001. What should they do? Draft: Review the recall code, affected product, and official consumer guidance. Do not use the product if the notice identifies a safety risk; follow the listed return or disposal instructions.

Stage	Score	Brief reason
Plan Quality	1	The response does not directly answer the user's request. It only outlines a process for generating a response rather than providing specific guidance on what the customer should do with the frozen shrimp and recall code. This makes it non-responsive and unsafe.
Writer Helpfulness	3	The response is generally correct and provides actionable guidance, but it lacks specific details about the recall code, affected product, or official consumer guidance. It is too vague and does not directly address the customer's situation with the specific recall code provided.
Final Helpfulness	3	The response is generally correct and provides actionable guidance, but it lacks specific details about the recall code REC-3539-00001 and does not mention contacting the manufacturer or checking official recall notices, which are critical steps for ensuring safety and proper action.

Final minus draft = +0. Reviewer preserved quality; no measured improvement was observed. The honest result is no measured improvement; the Reviewer preserved the Writer score rather than improving it.

11. Evaluation dashboard evidence

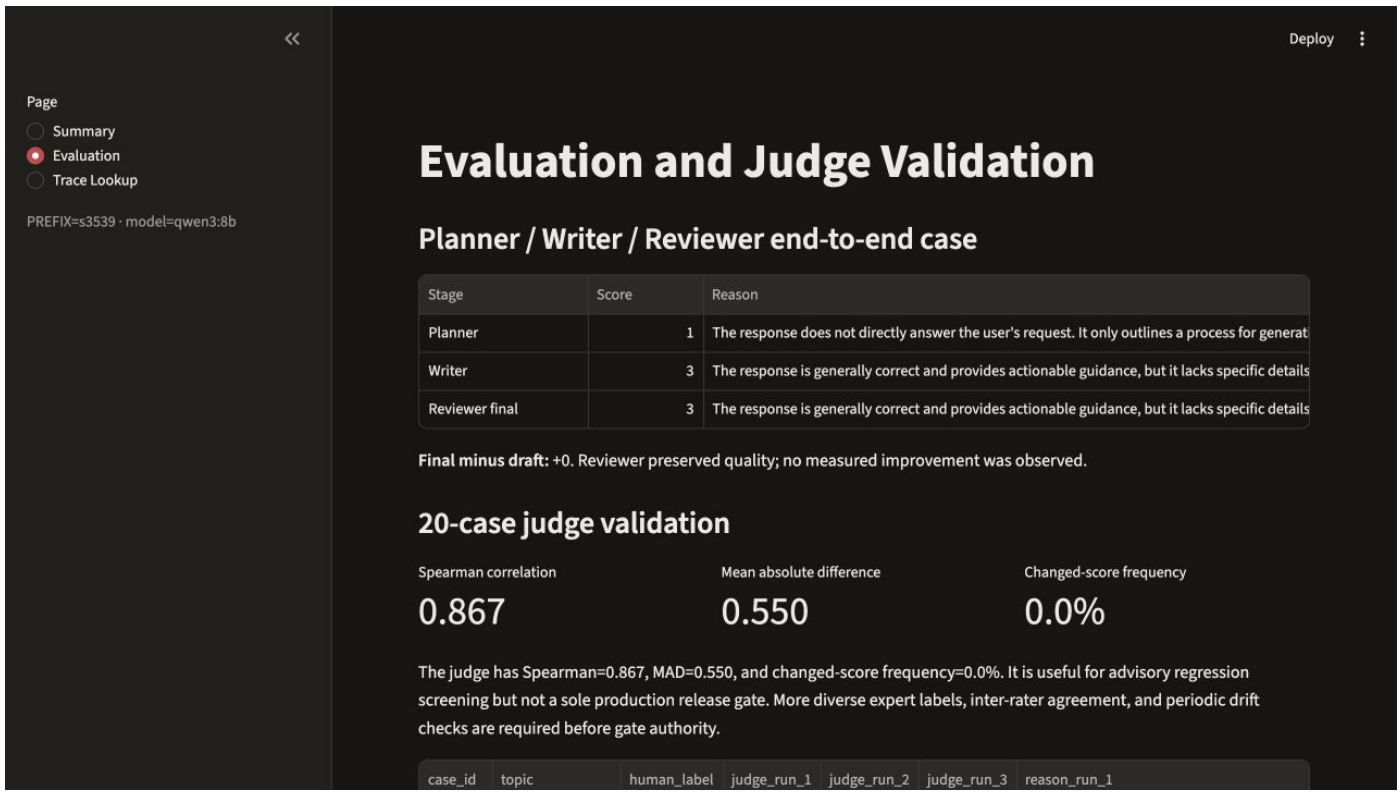


Figure 5. Real Streamlit Evaluation page with stage scores Spearman MAD repeatability and deployment-gate interpretation.

12. Frozen validation set and chronology

The 20 Writer drafts and hand-assigned labels were committed at `43ee393a839d19b97cbb16f5961b89eb2337869e` before judge implementation/results commit `78de14095a323861bbb6e92b3c829ae63cf4e307`. The labels were not generated by an LLM and were not changed after scoring.

Case	Topic	Human	Judge 1	Judge 2	Judge 3
recall-01	consumer_action	5	5	5	5
recall-02	record_matching	5	4	4	4
recall-03	allergen	5	5	5	5
recall-04	response_quality	5	4	4	4
recall-05	risk_guidance	4	5	5	5
recall-06	missing_data	4	4	4	4
recall-07	return_guidance	4	3	3	3
recall-08	non_match	4	3	3	3
recall-09	consumer_action	3	3	3	3
recall-10	record_matching	3	1	1	1

Cases 1–10 of the frozen set. Each row retains its original domain topic and the three raw judge scores.

The raw JSON keeps the original question draft metadata and run-1 reason for every case. The CSV contains exactly one manually assigned integer label per case.

13. Raw judge results — cases 11–20

Case	Topic	Human	Judge 1	Judge 2	Judge 3
recall-11	allergen	3	2	2	2
recall-12	response_quality	3	3	3	3
recall-13	risk_guidance	2	1	1	1
recall-14	missing_data	2	2	2	2
recall-15	return_guidance	2	2	2	2
recall-16	non_match	2	1	1	1
recall-17	consumer_action	1	1	1	1
recall-18	record_matching	1	1	1	1
recall-19	allergen	1	1	1	1
recall-20	risk_guidance	1	2	2	2

Validation statistic	Result
Spearman rank correlation	0.867
Mean absolute difference	0.550
Repeat runs	3 with identical settings
Changed cases / frequency	0 / 0.0%
Mean / max absolute change	0.000 / 0.000

The judge has Spearman=0.867, MAD=0.550, and changed-score frequency=0.0%. It is useful for advisory regression screening but not a sole production release gate. More diverse expert labels, inter-rater agreement, and periodic drift checks are required before gate authority.

14. Fifty confirmed cache misses and stage latency

Required check	Observed
New intentionally uncached questions	50
Requests served from cache	0
Cache-miss E2E p50 / p95	1.721 / 2.165 ms
Cache-hit trace	Reported separately: trace-cache-hit-3539

Stage	p50 ms	p95 ms	Mean ms	% miss E2E
cache_lookup	0.001	0.001	0.001	0.06%
planner	0.029	0.039	0.031	1.74%
writer	0.038	0.056	0.040	2.29%
reviewer	0.036	0.047	0.041	2.34%
queue_wait	0.636	0.643	0.633	35.89%
evaluator	0.026	0.036	0.028	1.61%

Optimize queue_wait first because it has the largest measured mean share. The cache-hit path is reported separately and is not mixed into these miss percentiles.

The 50-request load experiment uses the frozen bounded evaluator replay for comparable stage timing. The real local GEval latency is retained separately in trace-live-eval-3539 and is visibly 29.7 seconds for three sequential model judgments.

15. Experiment output

```
Vedant Vartak — DATA 260 Homework 10 experiment
PREFIX=s3539  MODEL=qwen3:8b  VERIFY_SEED=263539
PASS frozen judge cases: 20; repeat runs: 3
PASS Spearman: 0.867
PASS mean absolute difference: 0.550
PASS changed-score frequency: 0.0%
PASS confirmed cache misses: 50
PASS cache-served count during miss run: 0
PASS miss E2E p50/p95: 1.721 / 2.165 ms
PASS dominant stage: queue_wait
dataset_commit=43ee393a839d19b97cbb16f5961b89eb2337869e
```

Figure 6. Frozen experiment summary generated directly from evaluation_summary.json and stage_metrics.json.

16. Automated tests and verification

```
HW10 final verification
status=PASS
implementation_commit=78de14095a323861bbb6e92b3c829ae63cf4e307
resolved_hw10_tag=78de14095a323861bbb6e92b3c829ae63cf4e307
PASS ruff: True
PASS tests: ........                                                    [100%] | ==============================
PASS frozen_20_case_set
PASS three_repeat_runs
PASS human_dataset_precedes_scores
PASS fifty_confirmed_misses
PASS cache_hit_trace
PASS cache_miss_trace
PASS failed_trace
PASS single_trace_identifier
PASS dashboard_app_test
PASS judge_correlation_reported
PASS judge_mad_reported
```

Figure 7. Final verification: ruff passed 8 tests passed dashboard AppTest passed and all semantic checks passed.

Offline tests include:

configuration; 20-case dataset shape; required span fields; cache miss then hit; single trace ID and stage order; failed Writer trace; final-vs-draft scoring; and generated 50-miss artifacts.

verification.json stores SHA-256 digests for every raw artifact. The dashboard AppTest reported title Recall Pipeline Operations and zero exceptions.

17. Reproduction and artifact manifest

```
.venv/bin/pip install -r requirements-hw10.txt
.venv/bin/python scripts/run_hw10_experiments.py
PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 .venv/bin/python -m pytest -q tests/test_hw10.py
.venv/bin/python scripts/verify_hw10.py
PYTHONPATH=. .venv/bin/streamlit run code/hw10/dashboard.py --server.port 8510
```

Artifact	SHA-256
end_to_end_case.json	fb403348cc8d9a763e8d8ae1b9cbf75291176520c7c18f873f72913c00631e4a
evaluation_summary.json	5a79d188b610df2e1bcffbe9ccaf2cb1a74d40e0b64954b3bae86118d7da9c0d
judge_validation.json	e3564356bc13027d2cfc9f4f68e5bbf420638a87cc3de6b014440b234a4f797e
requests.jsonl	49887773a4ce7c062d09fba480767b1b4e3556d1a6305b4a6b5204a5490c276b
spans.jsonl	4af35269437d07e6e9b459cf7adee8dc81e0a01ae1b70035530b01c275042ae3
stage_metrics.json	d6801a41e5de0a3f8e4c52622796b0458b55700cc64d83000b8c79f47b83a546
trace_examples.json	58083a782d946213a896ecb1651d3b65b10f7a3cc484f31ff0005006f6028aec

The report is generated from these immutable raw files plus repository source. The human-label commit and final hw10 tag make the chronology independently verifiable.

18. Interpretation limitations and AI use

Interpretation

The Planner score of 1 reveals a coordination weakness: its outline is useful to the Writer but is not itself a user-facing answer. Writer and Reviewer both scored 3 so the Reviewer did not measurably improve this case. The judge aligns well with human ordering but MAD 0.55 shows that exact-score disagreement remains. Zero score changes under identical deterministic settings establish repeatability for this frozen set but do not prove robustness to prompt or model drift.

Limitations and next evidence

The in-memory Kafka analogue is used for deterministic offline replay while preserving the inherited s3539 topic journey. Before using the judge as a production gate I would collect multiple expert labels measure inter-rater agreement add adversarial and edge cases and rerun validation after every model or rubric change. Queue wait dominates the lightweight 50-miss replay and should be optimized first; real model evaluation latency is much larger and should remain asynchronous in production.

AI-use disclosure

I used an AI assistant to help debug code structure tests and report presentation. I checked the generated implementation and reran the saved experiments and verification commands. The 20 human labels were assigned before judge scoring and were not generated or changed by an LLM. The report identifies the local model raw artifacts and limitations so the evidence can be audited.

Final checklist

Item	Status
Four graded parts documented	Complete
20 cases × 3 judge runs	Complete
Hit miss failed traces	Complete
50 confirmed misses	Complete
Report length	Below the 35-page limit
Submission filename	Vartak_HW10.pdf